HW 2-3 | GPU Parallelized Particle Simulation

Jaeyoon Jung, Brandon Ton, Ben Krohn-Hansen | CMPSCI 267 | S25

INTRODUCTION

The main difference between assignment 2-3 and previous iterations of homework 2 is the method of simulation of particle interactions. In assignment 2-3, we utilize a GPU, which introduces challenges due to the increasing necessity of efficient neighbor searches and careful synchronization among relatively lightweight threads. Our implementation focuses on a system in which particles interact via short-range repulsive forces. The goal of this assignment is to achieve relatively linear scaling by reordering particles into spatial bins that align with cutoff distance in hopes of reducing the number of unnecessary force computations. Some key features of our implementation include removing ghost particle computations, changing bin size, and utilizing Thrust's vectors and scan operations rather than manually managing memory.

IMPLEMENTATION APPROACH

GPU synchronization is essential to ensure that computations occur in the correct order, especially considering that multiple kernels can operate on shared data. In our implementation, synchronization is maintained through a couple of unique mechanisms.

Our simulation workflow follows a fixed sequence. First, we compute the number of particles in each bin using compute_parts_per_bin(). Next, we compute bin start positions using thrust::exclusive_scan(). Following this, we sort particles into bins with assign_parts_to_bins(), compute forces, and finally update the position of molecules with move_gpu(). Each kernel launch is paired with a call to cudaDeviceSynchronize(). The purpose of this synchronization is to ensure that the preceding computations after each kernel launch (or groups of related kernels) complete before the next set of computations begins. While introducing cudaDeviceSynchronize() introduces small overhead (resulting in slightly less efficient code), this technique helps guarantee that the parts-per-bin counts and bin offsets are correct before we utilize the sorted particle array in separate force computations.

We also employ atomic operations in several key functions. In both compoute_parts_per_bin() and assign_parts_to_bins(), we use atomicAdd() to safely increment the particle count in each bin. Additionally, we determine the correct position for each particle in the sorted array. This helps reduce race conditions, since multiple threads concurrently update shared counters. By implementing atomic operations, we ensure that every thread writes its computed value to global memory without overwriting other threads' contributions.

A couple of Thrust library operations were also utilized in our implementation, which differs compared to previous assignments. Thrust::exclusive_scan() computes the exclusive prefix sum of particle counts with built-in synchronization, ensuring that the bin_offsets array reflects the starting indices for each bin in the sorted particle array. We also utilize Thrust device vectors to help with memory allocation and deallocation, rather than manually performing these tasks. This ensures that there are less manual errors when handling memory, and that data is clean and ready for subsequent kernels for calculations.

DESIGN

Our design evolved from an initial implementation that performed calculations on ghost particles to a more efficient binning strategy that better aligns with the CUDA execution model.

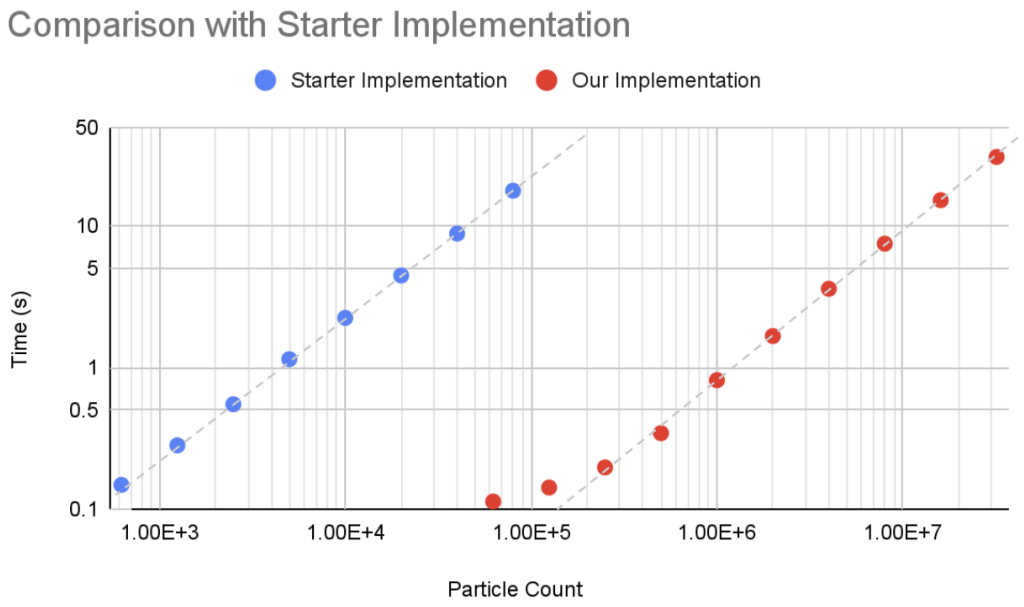
Regarding binning strategy and spatial decomposition, we set the bin size equal to the cutoff distance. This decision means that each bin contains only particles that can interact with each other, and each particle only needs to check the forces from particles in the neighboring bins and its own bin. This reduces overhead from excess calculations and assists with memory access. Unlike our MPI design that required calculations of ghost particles to handle boundary processes, CUDA allows all threads to access a shared global memory space. Due to this, our binning implementation eliminates the need for ghost particles, since adjacent bins are directly accessible by any thread. This greatly simplifies our implementation by reducing excess force calculations that previously led to distance errors in correctness checks.

Within the force computation force computation kernel, forces are accumulated in local variables and stored in registers before writing the final acceleration values back to global memory. The process of local force accumulation minimizes rounding errors while also avoiding the overhead of multiple global memory writes. We also reset each particle's acceleration before accumulating new forces to ensure that forces from previous simulation steps do not carry over. The Thrust library was also utilized for efficient rebinning. Exclusive_scan() computed bin start positions, which replaces manual CUDA implementations. Thrust device vectors also help manage memory automatically, and ensure that our binning data structures (such as parts_per_bin, bin_offsets, and sorted_parts) are allocated and updated consistently. Utilizing Thrust libraries improves performance as well as simplifies code. We also implement a 2D grid execution for our force computations. In our 2D grid, each thread corresponds to a specific bin in the simulation space. The 2D grid mapping improves on locality since each thread works on particles that are close in space. In addition, each thread examines only the particles in its bin and the eight neighboring bins, which greatly reduces the number of pairwise comparisons and avoids negligible force computations entirely.

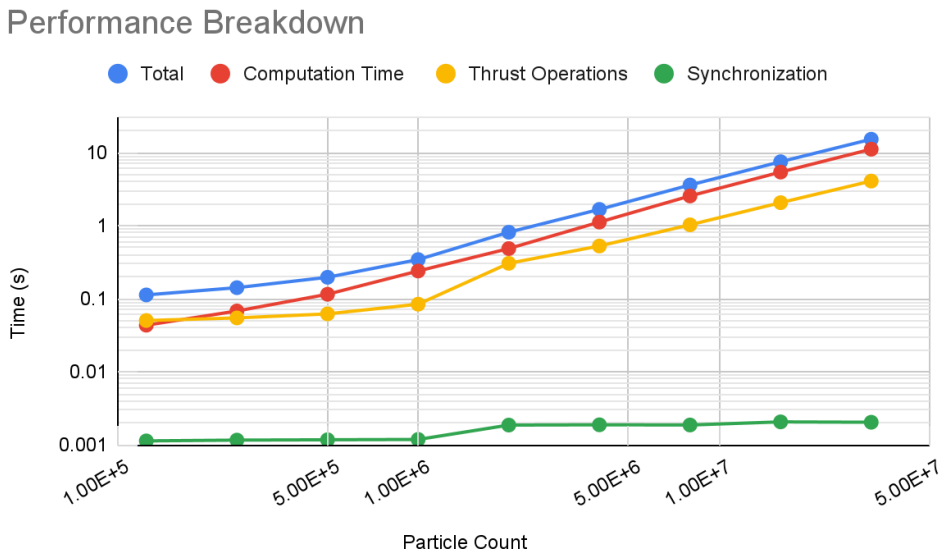
For parallelization techniques, each thread processes one particle to update the parts_per_bin array via atomic operations. To compute bin offsets, we utilize Thrust's exclusive scan to compute the starting index of each bin in the sorted array. Each thread then assigns a particle to its sorted position within the bin utilizing atomic increments. The kernel then iterates over bins in a 2D grid, accumulating force for each particle only from its bin and neighboring bins. Particles' movements are then updated using Velocity Verlet integration. Boundary conditions are also applied to reflect particles that move outside the simulation step.

The design of our implementation went through some updates as we continued development. Initially, our design attempted to implement ghost particles to handle interactions near domain boundaries. However, we recognized that this approach led to a higher computational overhead, and resulted in some inaccuracies in force calculations, which resulted in failure of the distance assertion statements in the correctness check. We then decided to switch to a binning strategy that leverages CUDA's shared global memory, which was then utilized to directly access neighboring particles without having ghost data. This technique not only decreased computational overhead, but also improved on our accuracy of force calculations, eliminating the average distance error.

SCALING AND RUNTIME

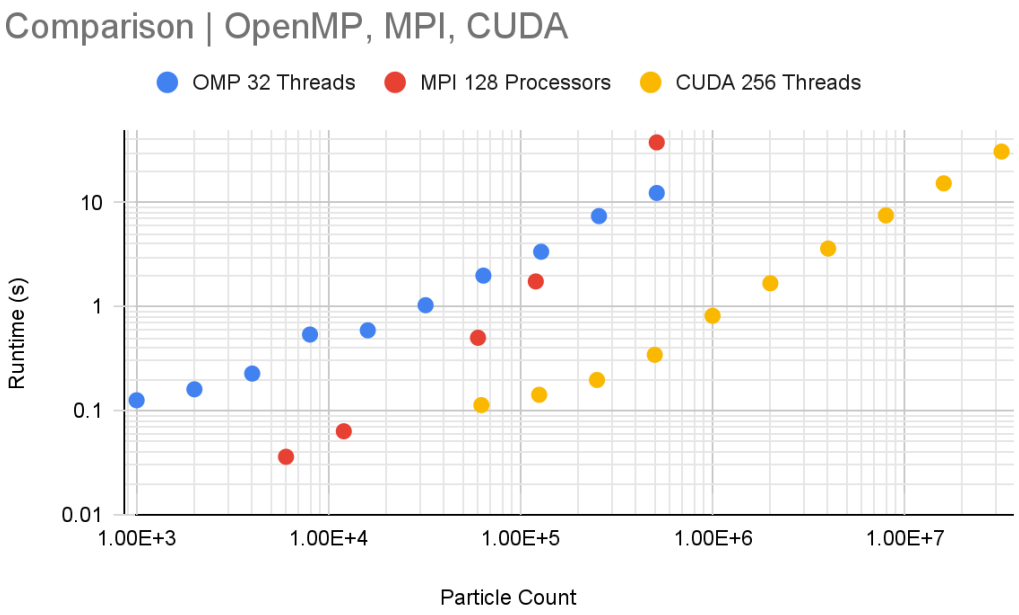


This graph compares the starter implementation (blue) and Our Implementation (red) across particle counts. Our implementation handles over two orders of magnitude more particles at similar runtimes and shows very linear scaling at higher particle counts.



This graph illustrates the performance breakdown of the simulation as the particle count increases. Computation time consists of assigning particles to bins, computing forces, and updating positions. Thrust operations involve setting up device vectors and performing an exclusive scan to organize particles efficiently. While exclusive scan is the primary contributor to Thrust runtime, it scales well, preventing Thrust operations from dominating the total runtime. As particle count increases, direct computation overshadows these preparatory steps, making computation time the primary

bottleneck. Above around 100,000 particles, the total runtime scales almost directly with direct computation time. The synchronization curve records the time spent in cudaDeviceSynchronize in main and is completely negligible with the overall runtime, suggesting good load balancing.



This graph compares the performance of OpenMP, MPI, and CUDA implementations across different particle counts. The OMP implementation exhibits a steady, nearly linear increase in runtime as the particle count grows. The MPI implementation, while initially outperforming OMP at lower particle counts, demonstrates a less predictable scaling behavior, likely due to the overhead of complicated memory transfers. The CUDA implementation shows an extremely linear and consistent runtime trend, maintaining significantly better performance—by several orders of magnitude—compared to the other two approaches.

INDIVIDUAL CONTRIBUTIONS:

Ben: Collecting and graphing all performance data. Runtime breakdown into computation, thrust operations, and synchronization. Performance section of write-up. Editing write-up.

Brandon: Program design, CUDA optimizations (Thrust library, memory access optimization), code restructuring (ghost particle changes), correctness check debugging, implementation write-up, design write-up

Jae: CUDA implementation with ghost particle, CUDA implementation with binning, correctness check, debugging, design and issue write-up